

Computer Organization and Architecture (B24CI0401)

UNIT -1

Basic Structure of Computers & Basic Processing

Basic Structure of Computers & Basic Processing: Computer Types, Functional Units, Input Unit, Memory Unit, Arithmetic and Logic Unit, Output Unit, Control Unit, Basic Operational Concepts. Memory Locations and Address, Byte addressability, Big-Endian and Little-Endian Assignments, Word Alignment, Accessing Numbers and Characters, Memory Operations, Instruction Execution and straight Line sequencing, Register Transfer Notation, RISC and CISC Instruction Sets, Instruction Execution and Straight-Line Sequencing, Branching, Generating Memory Address, Some Fundamental Concepts, Instruction Execution, Load Instructions, Arithmetic and Logic Instructions, Store Instructions.

Computer

A computer can be defined as a fast electronic calculating machine that accepts the (data) digitized input information process it as per the list of internally stored instructions and produces the resulting information. List of instructions are called programs & internal storage is called computer memory.

Computer types

- Digital computers have evolved into many different types that vary widely in size, cost, computational power, and intended use.
- Modern computers can be divided roughly into four general categories:

1. Embedded computers

- Are integrated into a large device or system in order to automatically monitor and control a physical process or environment.
- They are used for a specific purpose rather than for general processing tasks.
- Typical applications include industrial and home automation, appliances, telecommunication products, and vehicles.

2. Personal computers

- Widespread use in homes, educational institutions, and business and engineering office settings, primarily for dedicated individual use.
- They support a variety of applications such as general computation, document preparation, computer-aided design, audiovisual entertainment, interpersonal communication, and Internet browsing.
- Classifications for personal computers:

- Desktop - serve general needs and fit within a typical personal workspace.
- Workstation - offer higher computational capacity and more powerful graphical display capabilities for engineering and scientific work.
- Portable and Notebook computers - provide the basic features of a personal computer in a smaller lightweight package. They can operate on batteries to provide mobility.

3. Servers and Enterprise systems

- Large computers that are meant to be shared by a potentially large number of users who access them from some form of personal computer over a public or private network.
- May host large databases and provide information processing for a government agency or a commercial organization.

4. Supercomputers and Grid computers

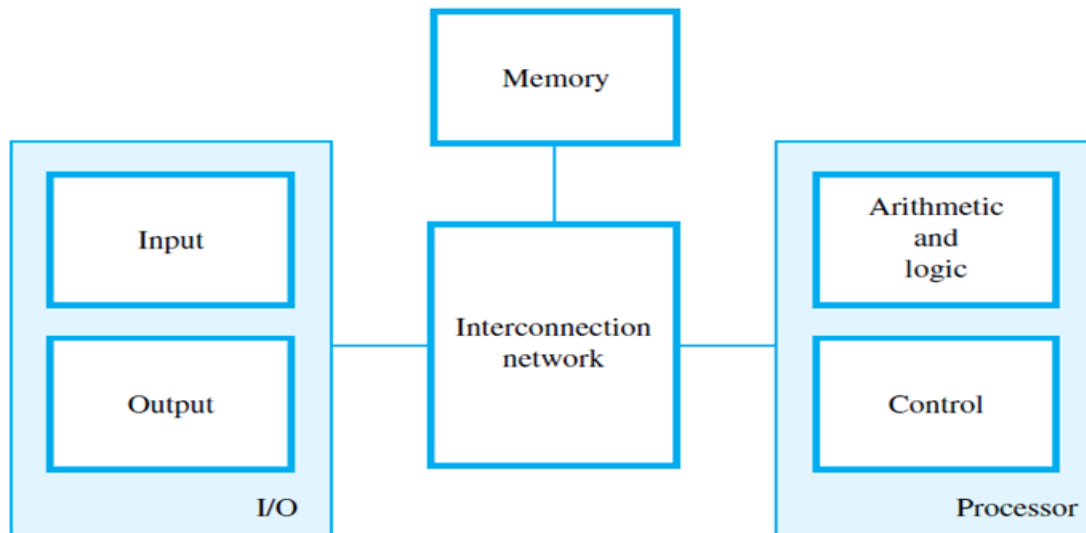
- Offer the highest performance.
- Most expensive and physically the largest category of computers.
- Used for the highly demanding computations needed in weather forecasting, engineering design and simulation, and scientific work.
- They have a high cost.

Grid computers provide a more cost-effective alternative.

- Combine a large number of personal computers and disk storage units in a physically distributed high-speed network, called a grid, which is managed as a coordinated computing resource.
- By evenly distributing the computational workload across the grid, it is possible to achieve high performance on large applications ranging from numerical computation to information searching.

Functional Units of Computer:

A computer consists of five functionally independent main parts: input, memory, arithmetic and logic, output, and control units, as shown in below Figure.



Basic functional units of a computer.

The input unit accepts coded information from human operators using devices such as keyboards, or from other computers over digital communication lines. The information received is stored in the computer's memory, either for later use or to be processed immediately by the arithmetic and logic unit.

The processing steps are specified by a program that is also stored in the memory. Finally, the results are sent back to the outside world through the output unit. All these actions are coordinated by the control unit. An interconnection network provides the means for the functional units to exchange information and coordinate their actions.

Arithmetic and logic circuits, in conjunction with the main control circuits, as the processor. Input and output equipment is often collectively referred to as the input-output (I/O) unit. We now take a closer look at the information handled by a computer. It is convenient to categorize this information as either instructions or data. Instructions, or machine instructions, are explicit commands that govern the transfer of information within a computer as well as between the computer and its I/O devices specify the arithmetic and logic operations to be performed.

A program is a list of instructions which performs a task. Programs are stored in the memory. The processor fetches the program instructions from the memory, one after another, and performs the desired operations. The computer is controlled by the stored program, except for possible external interruption by an operator or by I/O devices connected to it. Data are numbers and characters that are used as operands by the instructions. Data are also stored in the memory.

The instructions and data handled by a computer must be encoded in a suitable format.

Most present-day hardware employs digital circuits that have only two stable states. Each instruction, number, or character is encoded as a string of binary digits called bits, each having one of two possible values, 0 or 1, represented by the two stable states. Numbers are usually represented in positional binary notation; Alphanumeric characters are also expressed in terms of binary codes.

Input Unit:

Computers accept coded information through input units. The most common input device is the keyboard. Whenever a key is pressed, the corresponding letter or digit is automatically translated into its corresponding binary code and transmitted to the processor.

Many other kinds of input devices for human-computer interaction are available, including the touchpad, mouse, joystick, and trackball. These are often used as graphic input devices in conjunction with displays. Microphones can be used to capture audio input which is then sampled and converted into digital codes for storage and processing.

Similarly, cameras can be used to capture video input. Digital communication facilities, such as the Internet, can also provide input to a computer from other computers and database servers.

Memory Unit:

A memory or storage unit is a hardware component which is used to store the data and instructions and retrieve it whether is required. Majority computer memory has categorized as temporary (RAM) and permanent memory (secondary memory). RAM is used for short-term, fast data access and essential for active program execution. On the other hand, storage or secondary memory provides permanent data storage. Hence, memory and storage units both are critical components of computer system. There are two classes of storage, called primary and secondary.

Primary Memory:

Primary memory, also called main memory, is a fast memory that operates at electronic speeds. Programs must be stored in this memory while they are being executed. The memory consists of many semiconductor storage cells, each capable of storing one bit of information. These cells are rarely read or written individually. Instead, they are handled in groups of fixed size called words. The memory is organized so that one word can be stored or retrieved in one basic operation. The number of bits in each word is referred to as the word length of the computer, typically 16, 32, or 64 bits. To provide easy access to any word in the memory, a distinct address is associated with each word location. Addresses are consecutive numbers, starting from 0, that identify successive locations. A particular word is accessed by specifying its address and issuing a control command to the memory that starts the storage or retrieval process. Instructions and data can be written into or read from the memory under the control of the processor. It is essential to be able to access any word location in the memory as quickly as possible. A memory in which any location can be accessed in a short and fixed amount of time after specifying its address is called a random-access memory (RAM). The time required to access one word is called the memory access time. This time is independent of the location of the word being accessed. It typically ranges from a few nanoseconds (ns) to about 100 ns for current RAM units.

Cache Memory:

As an adjunct to the main memory, a smaller, faster RAM unit, called a cache, is used to hold sections of a program that are currently being executed, along with any associated data. The cache is tightly coupled with the processor and is usually contained on the same integrated-circuit chip. The purpose of the cache is to facilitate high instruction execution rates.

At the start of program execution, the cache is empty. All program instructions and any required data are stored in the main memory. As execution proceeds, instructions are fetched into the processor chip, and a copy of each is placed in the cache. When the execution of an instruction requires data located in the main memory, the data are fetched and copies are also placed in the cache.

Now, suppose several instructions are executed repeatedly as happens in a program loop. If

these instructions are available in the cache, they can be fetched quickly during the period of repeated use. Similarly, if the same data locations are accessed repeatedly while copies of their contents are available in the cache, they can be fetched quickly.

Secondary Storage:

Although primary memory is essential, it tends to be expensive and does not retain information when power is turned off. Thus additional, less expensive, permanent secondary storage is used when large amounts of data and many programs must be stored, particularly for information that is accessed infrequently. Access times for secondary storage are longer than for primary memory. A wide selection of secondary storage devices is available, including magnetic disks, optical disks (DVD and CD), and flash memory devices.

Arithmetic and Logic Unit:

Most computer operations are executed in the arithmetic and logic unit (ALU) of the processor. Any arithmetic or logic operation, such as addition, subtraction, multiplication, division, or comparison of numbers, is initiated by bringing the required operands into the processor, where the operation is performed by the ALU. For example, if two numbers located in the memory are to be added, they are brought into the processor, and the addition is carried out by the ALU. The sum may then be stored in the memory or retained in the processor for immediate use. When operands are brought into the processor, they are stored in high-speed storage elements called registers. Each register can store one word of data. Access times to registers are even shorter than access times to the cache unit on the processor chip.

Functions of an ALU

.Arithmetic operations:- The ALU can perform basic operations such as addition, subtraction, multiplication, and division.

Logic Operations:- The ALU can also perform logical operations like AND, OR, NOT, XOR and bit-shifting operations.

Comparison:- The ALU can compare two binary values and determine whether they are equal, greater than or less than each other. This function is most widely used in programming and sorting algorithms.

Output Unit:

The output unit is the counterpart of the input unit. Its function is to send processed results to the outside world. A familiar example of such a device is a printer. Most printers employ either photocopying techniques, as in laser printers, or ink jet streams. Such printers may generate output at speeds of 20 or more pages per minute. However, printers are mechanical devices, and as such are quite slow compared to the electronic speed of a processor.

Some units, such as graphic displays, provide both an output function, showing text and graphics, and an input function, through touchscreen capability. The dual role of such units is the reason for using the single name input/output (I/O) unit in many cases.

Control Unit:

The memory, arithmetic and logic, and I/O units store and process information and perform input and output operations. The operation of these units must be coordinated in some way. This is the responsibility of the control unit. The control unit is effectively the nerve center that sends control signals to other units and senses their states.

I/O transfers, consisting of input and output operations, are controlled by program instructions

that identify the devices involved and the information to be transferred. Control circuits are responsible for generating the timing signals that govern the transfers and determine when a given action is to take place. Data transfers between the processor and the memory are also managed by the control unit through timing signals. It is reasonable to think of a control unit as a well-defined, physically separate unit that interacts with other parts of the computer. In practice, however, this is seldom the case. Much of the control circuitry is physically distributed throughout the computer. A large set of control lines (wires) carries the signals used for timing and synchronization of events in all units.

Functions of Control Unit

- **Instruction Fetch** – To run and execute a program; a CU fetches instructions from RAM (Random Access Memory).
- **Instruction Decoding** – It decodes the fetched instructions to determine the operation to be performed.
- **Instruction Execution** – A CU executes the instructions by sending control signals to the appropriate functional units within the CPU, such as the ALU for arithmetic and logical operations
- **Control Flow Management** – The Control Unit is responsible for overseeing the control flow of the programme. It accomplishes this by updating the programme counter, which enables the CPU to go to the subsequent instruction in the sequence based on conditional statements or jumps.
- **Exception Handling** – The system effectively manages exceptions and interruptions, including hardware failures, system calls, and external events, by appropriately diverting the control flow of the central processing unit (CPU) to the planned procedure for managing such exceptions.
- **Pipeline Control (in pipelined CPUs)** – The modern CPU's are available with pipeline designs; a CU controls the steps of the pipeline and makes sure that instructions are processed quickly and safely.
- **Synchronization** – In the context of multi-core processors, a CU plays a crucial role in facilitating the coordination of instruction execution across several cores, hence guaranteeing the appropriate synchronization and maintenance of data consistency.

Basic Operational Concepts:

A computer system operates based on the interaction between the **processor (CPU)**, **memory (RAM/ROM)**, and **input/output devices**. The processor executes instructions stored in memory and coordinates all system activities.

We stated that the activity in a computer is governed by instructions. To perform a given task, an appropriate program consisting of a list of instructions is stored in the memory. Individual instructions are brought from the memory into the processor, which executes the specified operations. Data to be used as instruction operands are also stored in the memory.

(a) Instruction Execution Cycle

Every instruction in a program goes through a sequence of steps in the CPU:

1. **Fetch:** The CPU retrieves the instruction from memory.
2. **Decode:** The instruction is interpreted by the control unit.
3. **Execute:** The instruction is executed using ALU, registers, or memory.
4. **Store:** The result is written back to memory or a register.

(b) Data Flow in a Computer System

1. **Processor (CPU):** The brain of the computer that processes instructions.
2. **Memory (RAM, ROM):** Stores data and instructions temporarily (RAM) or permanently (ROM).
3. **Bus System:** Carries data between CPU, memory, and I/O devices.
 1. **Address Bus:** Specifies memory location.
 2. **Data Bus:** Transfers actual data.
 3. **Control Bus:** Carries control signals (read/write).
4. **I/O Devices:** Interact with external devices like keyboard, monitor, and storage.

A typical instruction might be

Load R2, LOC

This instruction reads the contents of a memory location whose address is represented symbolically by the label LOC and loads them into processor register R2. The original contents of location LOC are preserved, whereas those of register R2 are overwritten.

Execution of this instruction requires several steps.

First, the instruction is fetched from the memory into the processor. Next, the operation to be performed is determined by the control unit. The operand at LOC is then fetched from the memory into the processor.

Finally, the operand is stored in register R2. After operands have been loaded from memory into processor registers, arithmetic or logic operations can be performed on them. For example, the instruction

Add R4, R2, R3

Adds the contents of registers R2 and R3, then places their sum into register R4. The operands in R2 and R3 are not altered, but the previous value in R4 is overwritten by the sum.

After completing the desired operations, the results are in processor registers. They can be transferred to the memory using instructions such as

Store R4, LOC

This instruction copies the operand in register R4 to memory location LOC. The original contents of location LOC are overwritten, but those of R4 are preserved.

For Load and Store instructions, transfers between the memory and the processor are initiated by sending the address of the desired memory location to the memory unit and asserting the appropriate control signals. The data are then transferred to or from the memory.

Interfacing between processor and memory:

Figure 1.2 shows how the memory and the processor can be connected. It also shows some components of the processor that have not been discussed yet. The interconnections between these components are not shown explicitly since we will only discuss their functional characteristics here.

In addition to the ALU and the control circuitry, the processor contains a number of registers used for several different purposes. The instruction register (IR) holds the instruction that is

currently being executed. Its output is available to the control circuits, which generate the timing signals that control the various processing elements involved in executing the instruction. The program counter (PC) is another specialized register. It contains the memory address of the next instruction to be fetched and executed. During the execution of an instruction, the contents of the PC are updated to correspond to the address of the next instruction to be executed. It is customary to say that the PC points to the next instruction that is to be fetched from the memory. The Memory Address Register (MAR) holds the address of the location to or from which data are to be transferred. Memory Data Register (MDR) contains the data to be written into or read out of the addressed location. In addition to the IR and PC, Figure 1.2 shows general-purpose registers R_0 through R_{n-1} , often called processor registers. They serve a variety of functions, including holding operands that have been loaded from the memory for processing.

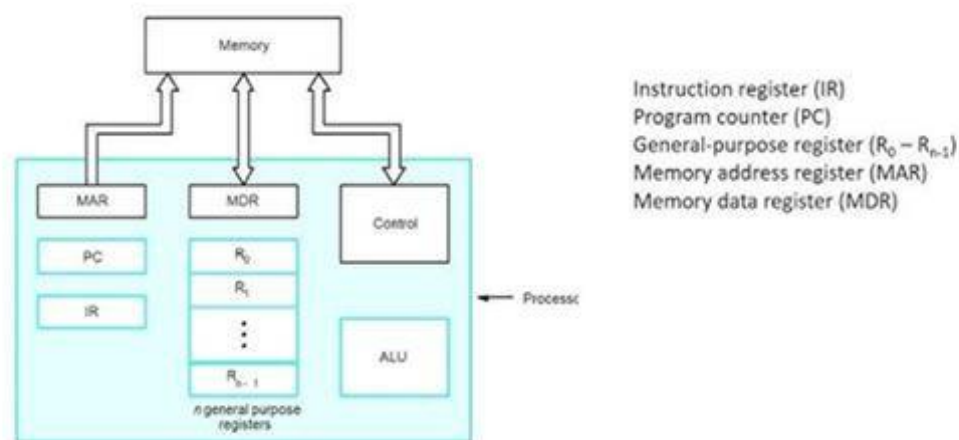


Figure 1.2. Connections between the processor and the memory.

The processor-memory interface is a circuit which manages the transfer of data between the main memory and the processor. If a word is to be read from the memory, the interface sends the address of that word to the memory along with a Read control signal. The interface waits for the word to be retrieved, then transfers it to the appropriate processor register. If a word is to be written into memory, the interface transfers both the address and the word to the memory along with a Write control signal.

Let us now consider some typical operating steps.

A program must be in the main memory for it to be executed. It is often transferred there from secondary storage through the input unit. Execution of the program begins when the PC is set to point to the first instruction of the program. The contents of the PC are transferred to the memory along with a Read control signal. When the addressed word (in this case, the first instruction of the program) has been fetched from the memory it is loaded into register IR. At this point, the instruction is ready to be interpreted and executed.

Instructions such as Load, Store, and Add perform data transfer and arithmetic operations.

If an operand that resides in the memory is required for an instruction, it is fetched by sending its address to the memory and initiating a Read operation. When the operand has been fetched from the memory, it is transferred to a processor register. After operands have been fetched in

this way, the ALU can perform a desired arithmetic operation, such as Add, on the values in processor registers. The result is sent to a processor register. If the result is to be written into the memory with a Store instruction, it is transferred from the processor register to the memory, along with the address of the location where the result is to be stored, then a Write operation is initiated.

At some point during the execution of each instruction, the contents of the PC are incremented so that the PC points to the next instruction to be executed. Thus, as soon as the execution of the current instruction is completed, the processor is ready to fetch a new instruction.

In addition to transferring data between the memory and the processor, the computer accepts data from input devices and sends data to output devices. Thus, some machine instructions are provided for the purpose of handling I/O transfers.

Normal execution of a program may be pre-empted if some device requires urgent service. For example, a monitoring device in a computer-controlled industrial process may detect a dangerous condition. To respond immediately, execution of the current program must be suspended. To cause this, the device raises an interrupt signal, which is a request for service by the processor. The processor provides the requested service by executing a program called an interrupt-service routine. Because such diversions may alter the internal state of the processor, its state must be saved in the memory before servicing the interrupt request. Normally, the information that is saved includes the contents of the PC, the contents of the general-purpose registers, and some control information. When the interrupt-service routine is completed, the state of the processor is restored from the memory so that the interrupted program may continue.

Instruction Set RISC and CISC

Two different approaches have been used in the design of instruction set architectures:

Reduced Instruction Set Computer (RISC)

Complex Instruction Set Computer (CISC)

RISC-Style Instruction Sets

The RISC approach is based on the observation that:

- Simple instructions can be executed very quickly.
- Complex instructions often require multiple cycles.
- A small number of frequently used instructions dominate program execution.

Main Characteristics of RISC

1. The instruction set is kept small.
2. Each instruction performs a simple operation.
3. Most instructions execute in one clock cycle.
4. A load/store architecture is used.
5. Only Load and Store instructions access memory.
6. Arithmetic and logic instructions operate only on registers.

7. Many general-purpose registers are provided.
8. Instructions are of fixed length.
9. Few addressing modes are supported.
10. Control is usually implemented using hardwired control.
11. The design supports efficient pipelining.

Examples of RISC processors:

- ARM (used in smartphones, IoT)
- MIPS (used in embedded systems)
- RISC-V(open-source CPU architecture)
- PowerPC

CISC Instruction Sets

CISC processors have complex and variable-length instructions that can perform many operations. The CISC approach was developed to reduce the semantic gap between high-level languages and machine instructions.

Main Characteristics of CISC

1. Many instructions are provided.
2. Instructions can perform complex operations.
3. Instructions may vary in length.
4. Many addressing modes are supported.
5. Memory-to-memory operations are allowed.
6. Some instructions require multiple cycles to execute.
7. Fewer general-purpose registers are typically used.
8. Control is often implemented using microprogramming.

RISC vs. CISC

CISC	RISC
Emphasis on hardware	Emphasis on software
Multiple instruction sizes and formats	Instructions of same set with few formats
Less registers	Uses more registers
More addressing modes	Fewer addressing modes
Extensive use of microprogramming	Complexity in compiler
Instructions take a varying amount of cycle time	Instructions take one cycle time
Pipelining is difficult	Pipelining is easy

Memory Locations and Addresses

Numbers and character operands, as well as instructions, are stored in the memory of a

computer. We will now consider how the memory is organized. The memory consists of many millions of storage cells, each of which can store a bit of information having the value 0 or 1. Because a single bit represents a very small amount of information, bits are seldom handled individually. The usual approach is to deal with them in groups of fixed size. For this purpose, the memory is organized so that a group of n bits can be stored or retrieved in a single, basic operation. Each group of n bits is referred to as a word of information, and n is called the word length. The memory of a computer can be schematically represented as a collection of words as shown in Figure 5.

Modern computers have word lengths that typically range from 16 to 64 bits. If the word length of a computer is 32 bits, a single word can store a 32-bit 2's-complement number or four ASCII characters, each occupying 8 bits, as shown in Figure 4. A unit of 8 bits is called a byte. Machines may require one or more words for representing fixed-point numbers or addresses.

To provide easy access to any word in the memory, a distinct address is associated with each word location. Addresses are numbers that identify successive locations. A given word is accessed by specifying its address and issuing a control command that starts the storage or retrieval process.

The number of bits in each word is often referred to as the word length of the computer. Typical word lengths range from 16 to 64 bits. The capacity of the memory is usually expressed in terms of the number of words it contains. The number of words is often a power of 2. For example, a 24-bit address generates an address space of 2^{24} (16,777,216) locations. This number is usually written as 16M (16 mega), where 1M is the number 2^{20} (1,048,576). A 32-bit address creates an address space of 2^{32} or 4G (4 giga) locations.

Other notational conventions that are commonly used are K (kilo) for the number 2^{10} (1,024), M (mega) for 2^{20} , G (giga) for 2^{30} , and T (tera) for 2^{40} .

Byte Addressability

We now have three basic information quantities to deal with: the bit, byte, and word. A byte is always 8 bits, but the word length typically ranges from 16 to 64 bits. It is impractical to assign distinct addresses to individual bit locations in the memory.

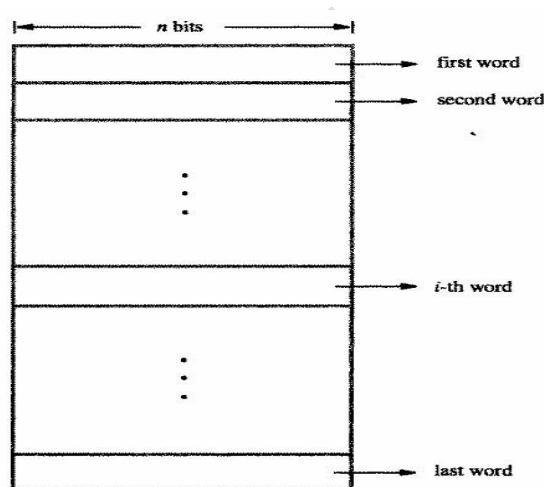


Fig 2: Memory word

The most practical assignment is to have successive addresses refer to successive byte locations in the memory. This assignment is used in most modern computers and is called byte-addressable memory. Byte locations have addresses 0,1,2,... Thus, if the word length of the machine is 32 bits, successive words are located at addresses 0,4,8,..., with each word consisting of four bytes.

Big-Endian and Little-Endian Assignments

There are two ways that byte addresses can be assigned across words as shown in fig 3.

- Big-endian assignment, When a word is given the same address as that of the most significant (the leftmost byte) byte of the word. This system is used in Motorola and PowerPC processor. Here, high-byte of data is stored at lower address and low-byte of data at higher address.
- Little-endian assignment, where the lower byte addresses are used for the less significant bytes (the rightmost) bytes of the word. This is used in Intel processors. Here, lower addresses contain low-order bytes of data.

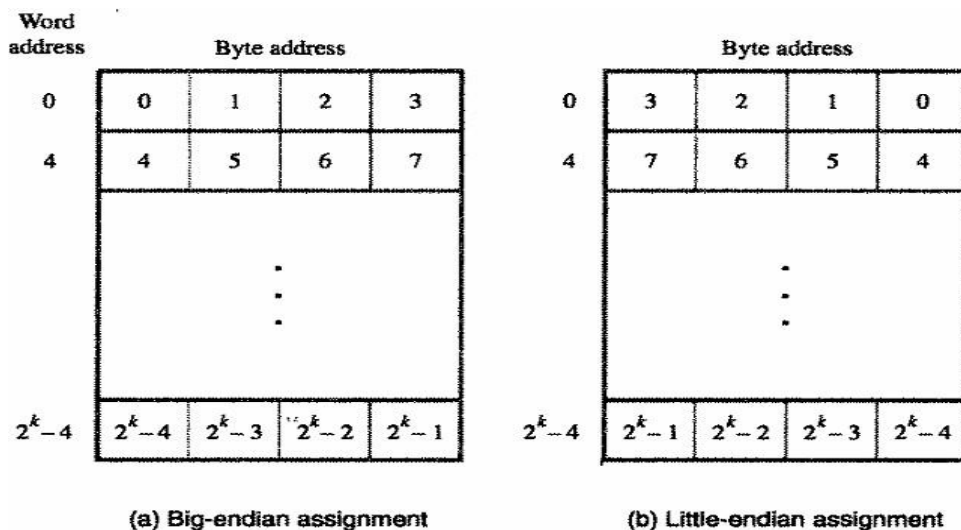


Fig3: Big endian and little-endian assignment

In addition to specifying the address ordering of bytes within a word, it is also necessary to specify the labelling of bits within a byte or word. The most common convention is to label bits from left to right.

Word Alignment

In the case of a 32-bit word length, natural word boundaries occur at addresses 0, 4, 8, ... as shown in Figure 3a & b. We say that the word locations have aligned addresses. In general, words are said to be aligned in memory if they begin at byte addresses that are multiples of the number of bytes in a word.

Ex: Data= 22354456 (4 bytes or 32-bit data)

Address

Address

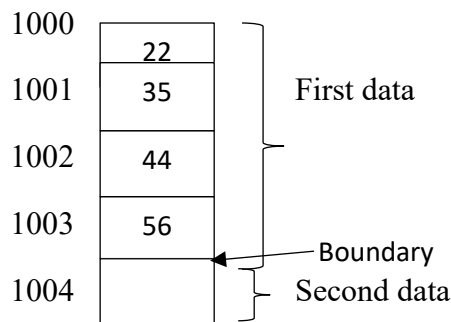
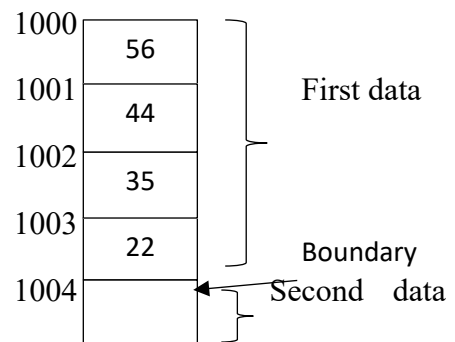


Fig.3a) Big-endian assignment



b) Little-endian assignment

Accessing Numbers, Characters, & Character Strings

A number usually occupies one word. It can be accessed in the memory by specifying its word address. Similarly, individual characters can be accessed by their byte address as shown in fig 4.

In many applications, it is necessary to handle character strings of variable length. Successive byte locations contain successive characters of the string. There are two ways to indicate the length of the string. A special control character may be inserted to indicate the end of the string, or a separate memory word location or processor register can contain a number indicating the length of the string in bytes.

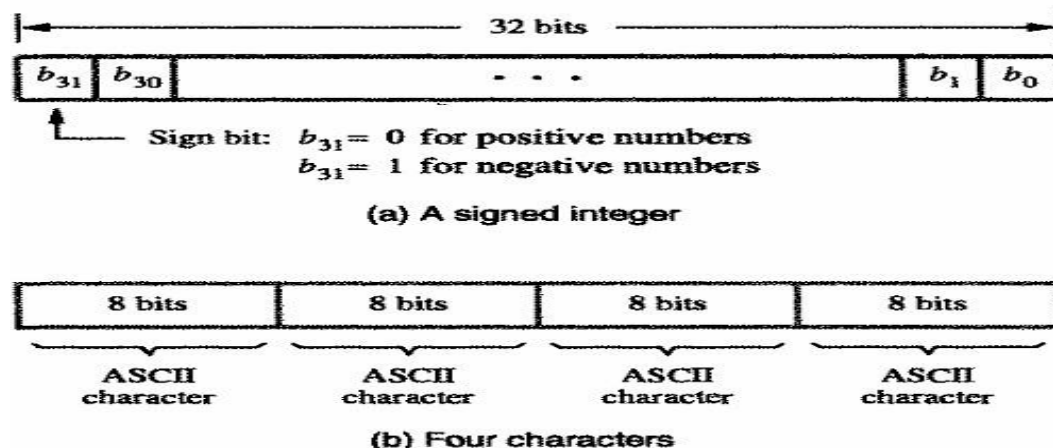


Fig4: Encoded information in a 32-bit word

Memory operations:

Instructions as well as the operands are stored in memory. To execute an instruction, the processor has to obtain the instruction and data from memory using control signals. The result of the operation may also be copied to the memory. Thus, two basic operations are performed with the memory.

a) **Load (Read or Fetch):** This operation transfers a copy of the contents of the specified memory location to the processor register. Contents of memory are not changed, but the previous content of the register is lost. The steps are:

- Processor sends the address of the desired memory location.
- Processor issues a Read control signal to memory to request the data.
- Memory sends the data to the processor

- Data gets stored in processor register
- b) **Store (Write):** This operation transfers the information from the processor register to the memory location specified in the instruction. This will destroy the original contents of that memory location. The steps are:
- Processor sends the address of the memory location where it wants to store data.
 - The processor issues a write signal
 - The content of the processor register is written into the specified memory location (data remains in the register as well)

Note: In both the operations, source contents are not modified.

Instructions and instruction sequencing

A computer must have instructions capable of performing four types of operations

- Data transfers between the memory and the processor registers
- Arithmetic and logic operations on data
- Program sequencing and control
- I/O transfers

Register Transfer Notation: -

Transfer of information from one location in the computer to another. Possible locations that may be involved in such transfers are memory locations that may be involved in such transfers are memory locations, processor registers, or registers in the I/O subsystem. Most of the time, we identify a location by a symbolic name standing for its hardware binary address.

Example, names for the addresses of memory locations may be LOC, PLACE, A, VAR2; processor registers names may be R0, R5; and I/O register names may be DATAIN, OUTSTATUS, and so on. The contents of a location are denoted by placing square brackets around the name of the location. Thus, the expression

$R1 \leftarrow [LOC]$

Means that the contents of memory location LOC are transferred into processor register R1. As another example, consider the operation that adds the contents of registers R1 and R2, and then places their sum into register R3. This action is indicated as

$R3 \leftarrow [R1] [R2]$

This type of notation is known as Register Transfer Notation (RTN). Note that the right-hand side of an RTN expression always denotes a value, and the left-hand side is the name of a location where the value is to be placed, overwriting the old contents of that location.

Assembly Language Notation: -

We need another type of notation to represent machine instructions and programs. For this, we use assembly language format. Here, we use English like words called mnemonics to represent

an operation uniquely.

Format-- Mnemonics operand1, operand2, operand3
(Op-code)

The (op-code) mnemonic is followed by at least 1-blank space or tab character. Then the information that specifies the operands is given.

Ex: Two-operand :- Load destination, source
Store source, destination

For example, an instruction that causes the transfer described above, from memory location LOC to processor register R1, is specified by the statement

Load R2, LOC

Store R2, LOC ; From processor register R1 to memory location LOC, is specified by the statement

Ex: Three-operand:- Add destination, source1, source2

The constant value if added to processor register R3 and result is store back to R3 specified by the statement

Add R3, #5

The second example of adding two numbers contained in processor registers R2 and R3 and placing their sum in R3 can be specified by the assembly language statement

Add R4, R2, R3

- *An instruction specifies an operation to be performed and the operands involved.*
- English words Load and Add denote the required operations.
- In the assembly-language instructions of actual (commercial) processors, such operations are defined by using *mnemonics, which are typically abbreviations of the words describing the operations.*
- The user program in its original alphanumeric text format is called a **Source program**, and the assembled machine-language program is called an **Object program**.

Basic Instructions: -

The operation of adding two numbers is a fundamental capability in any computer. The statement.

$$C = A + B$$

In a high-level language program is a command to the computer to add the current values of the two variables called A and B, and to assign the sum to a third variable, C. When the program containing this statement is compiled, the three variables, A, B, and C, are assigned to distinct locations in the memory. We will use the variable names to refer to the corresponding memory location addresses. The contents of these locations represent the values of the three variables. Hence, the above high-level language statement requires the action.

$$C \leftarrow [A] + [B]$$

To carry out this action, the contents of memory locations A and B are fetched from the memory and transferred into the processor where their sum is computed. This result is then sent back to the memory and stored in location C.

To carry out this kind of operation, assembly language provides 3-types of instructions format.

a) Three address instruction

The instruction has the format

Operation Destination, Source1, Source 2

Ex: Load R2, A ; Will copy the contents of memory location A into register R2
 Load R3, B ; Will copy the contents of memory location B into register R3
 Add R4, R3, R2 ; The content of R2 is adding with the content of R3 and result stored
 to register R4.
 Store R4, C ; Will transfer the contents of register R4 to memory location C

b) Two address instruction

These instructions specify two operands or addresses, which may be memory locations or registers. The instruction operates on the contents of both operands, and the result may be stored in the same or a different location.

The general format is

Operation Source, Destination

Ex: Load R2, A ; Will copy the contents of memory location A into register R2
 Add R3, R2 ; The content of R2 is adding with the content of R3 and result store back
 to register R3.
 Store R3, C ; Will transfer the contents of register R3 to memory location C

c) One address instruction

These instructions specify one operand or address, which typically refers to a memory location or register. The instruction operates on the contents of that operand, and the result may be stored in the same or a different location.

This uses an implied ACCUMULATOR register for data manipulation. One operand is in the accumulator and the other is in the register or memory location. Implied means that the CPU already knows that one operand is in the accumulator so there is no need to specify it.

The instructions has the format

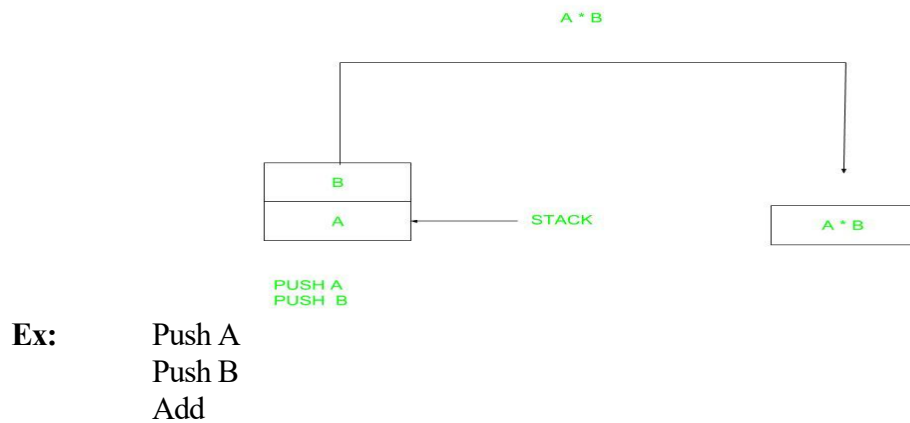
Operation Source/Destination

Ex: Load A ; Will copy the contents of memory location A into accumulator register
 Add B ; The content of accumulator register is adding with the content of memory
 location B and result stored in accumulator.
 Store C ; Will transfer the contents of accumulator to memory location C

d) Zero address instruction

These instructions do not specify any operands or address. Instead, they operate on data

stored in registers or memory locations implicitly defined by the instruction.



Instruction Execution and Straight-Line Sequencing: -

In the preceding discussion of instruction formats, we used to task

$$C \leftarrow [A] + [B]$$

fig 5 shows a possible program segment for this task as it appears in the memory of a computer. We have assumed that the computer allows one memory operand per instruction and has several processor registers. The three instructions of the program are in successive word locations, starting at location i . since each instruction is 4 bytes long, the second and third instructions start at addresses $i + 4$ and $i + 8$. Let us consider how this program is executed.

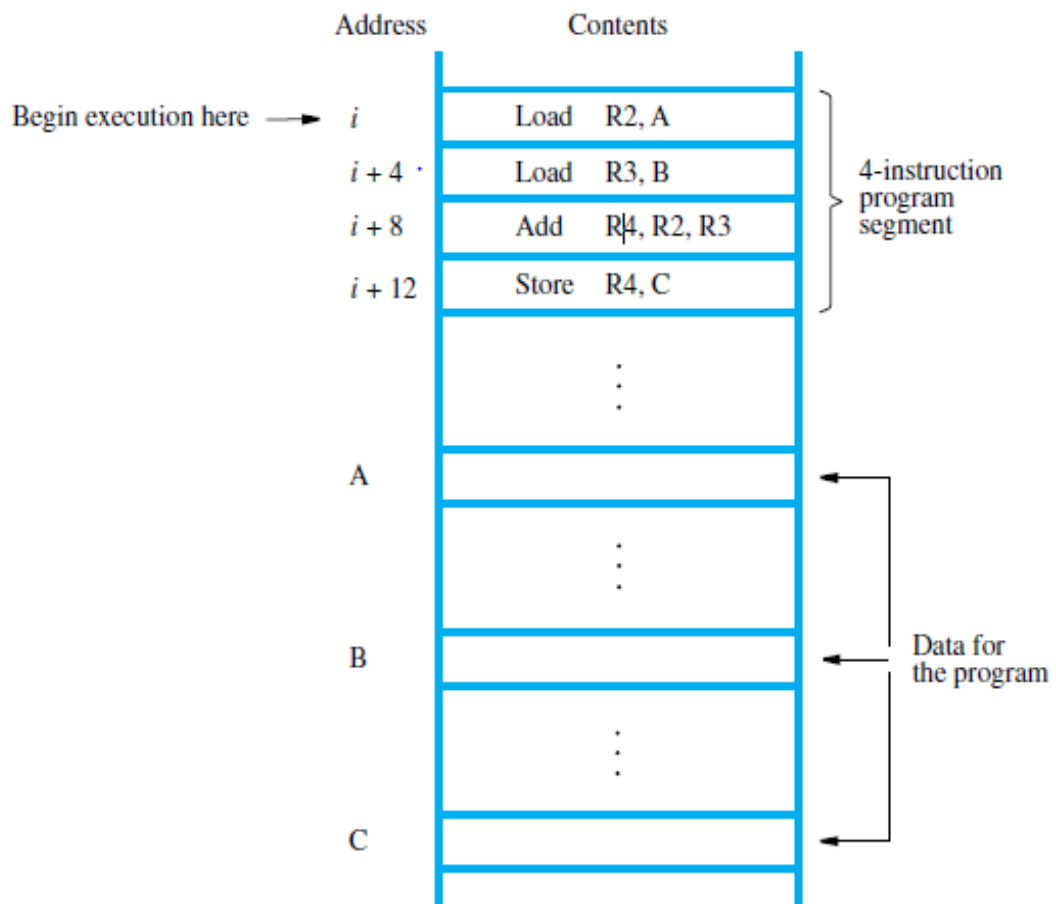


Fig 5. A program for $C \leftarrow [A] + [B]$

The processor contains a register called the program counter (PC), which holds the address of the instruction to be executed next. To begin executing a program, the address of its first instruction (i in our example) must be placed into the PC. Then, the processor control circuits use the information in the PC to fetch and execute instructions, one at a time, in the order of increasing addresses. This is called **straight-line sequencing**. During the execution of each instruction, the PC is incremented by 4 to point to the next instruction. Thus, after the Move instruction at location $i + 8$ is executed, the PC contains the value $i + 12$, which is the address of the first instruction of the next program segment. Executing a given instruction is a two-phase procedure. In the first phase, called instruction fetch, the instruction is fetched from the memory location whose address is in the PC.

This instruction is placed in the instruction register (IR) in the processor. The instruction in IR is examined to determine which operation is to be performed. The specified operation is then performed by the processor. This often involves fetching operands from the memory or from processor registers, performing an arithmetic or logic operation, and storing the result in the destination location.

Some Fundatmental Concepts: Main hardware components of a processor

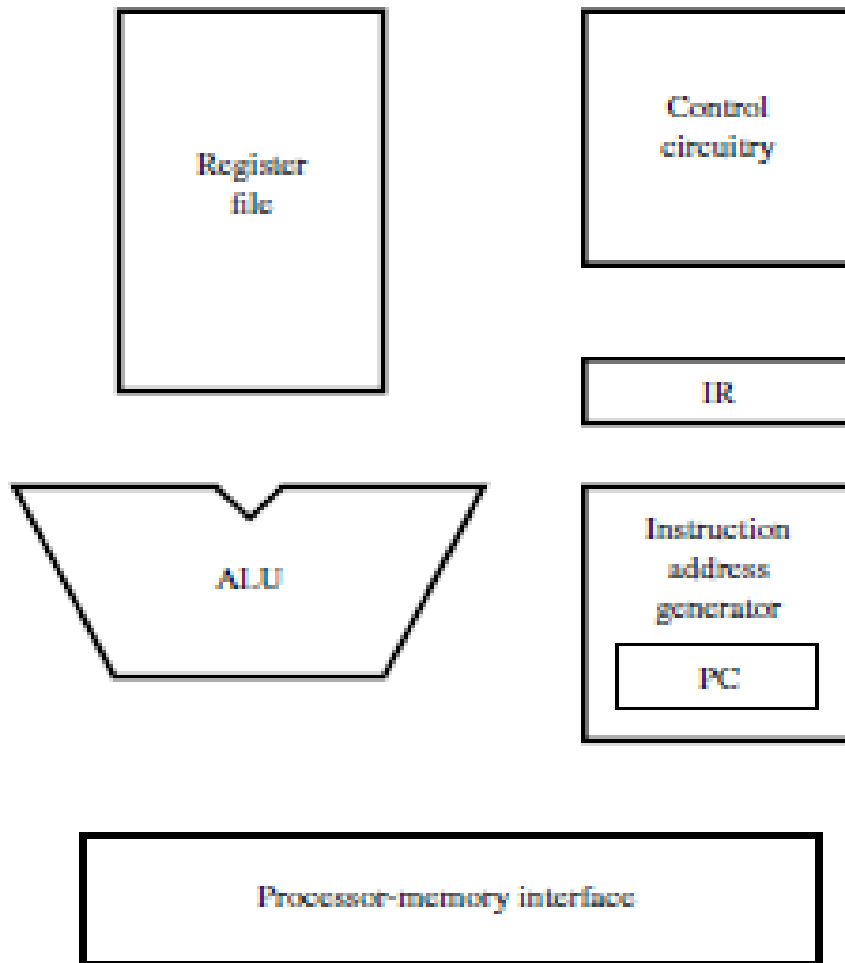


Fig 6. Main hardware components of a processor

Typical computing task consists of a series of operations specified by a sequence of machine-language instructions that constitute a program. The processor fetches one instruction at a time and performs the operation specified. Instructions are fetched from successive memory locations until a branch or a jump instruction is encountered. The processor uses the *program counter*, *PC*, to keep track of the address of the next instruction to be fetched and executed. After fetching an instruction, the contents of the PC are updated to point to the next instruction in sequence. A branch instruction may cause a different value to be loaded into the PC.

When an instruction is fetched, it is placed in the *instruction register*, *IR*, from where it is interpreted, or decoded, by the processor's control circuitry. The IR holds the instruction until its execution is completed.

Consider a 32-bit computer in which each instruction is contained in one word in the memory, as in RISC-style instruction set architecture.

To execute an instruction, the processor has to perform the following steps:

- 1) Fetch the contents of the memory location pointed to by the PC. The contents of this location are the instruction to be executed; hence they are loaded into the IR. In register transfer notation, the required action is $IR \leftarrow [[PC]]$
- 2) Increment the PC to point to the next instruction. Assuming that the memory is byte addressable, the PC is incremented by 4; that is $PC \leftarrow [PC] + 4$

3) Carry out the operation specified by the instruction in the IR.

Fetching an instruction and loading it into the IR is usually referred to as the *instruction fetch phase*. *Performing the operation specified in the instruction constitutes the instruction execution phase*.

With few exceptions, the operation specified by an instruction can be carried out by performing one or more of the following actions:

- Read the contents of a given memory location and load them into a processor register.
- Read data from one or more processor registers.
- Perform an arithmetic or logic operation and place the result into a processor register.
- Store data from a processor register into a given memory location.

The hardware components needed to perform these actions are shown in figure 6. The processor communicates with the memory through the processor-memory interface, which transfers data from and to the memory during Read and Write operations. The instruction address generator updates the contents of the PC after every instruction is fetched. The register file is a memory unit whose storage locations are organized to form the processor's general-purpose registers. During execution, the contents of the registers named in an instruction that performs an arithmetic or logic operation are sent to the arithmetic and logic unit (ALU), which performs the required computation. The results of the computation are stored in a register in the register file.

Instruction Execution:

Load Instructions

Consider the instruction Load R5, X(R7), which uses the Index addressing mode to load a word of data from memory location $X + [R7]$ into register R5. Execution of this instruction involves the following actions:

- Fetch the instruction from the memory.
- Increment the program counter.
- Decode the instruction to determine the operation to be performed.
- Read register R7.
- Add the immediate value X to the contents of R7.
- Use the sum $X + [R7]$ as the effective address of the source operand, and read the contents of that location in the memory.
- Load the data received from the memory into the destination register, R5.

Arithmetic and Logic Instructions

A typical instruction of this type is Add R3, R4, R5

It requires the following steps:

1. Fetch the instruction and increment the program counter.
2. Decode the instruction and read the contents of source registers R4 and R5.
3. Compute the sum $[R4] + [R5]$.
4. Load the result into the destination register, R3.

If the instruction uses an immediate operand, as in Add R3, R4, #1000

- The immediate value is given in the instruction word. Once the instruction is loaded into the IR, the immediate value is available for use in the addition operation.
- The same four step sequence can be used, with steps 2 and 3 modified as:
 2. Decode the instruction and read register R4.
 3. Compute the sum $[R4] + 1000$.

Store Instructions

For example, the instruction

Store R6, X(R8)

stores the contents of register R6 into memory location $X + [R8]$.

It can be implemented as follows:

1. Fetch the instruction and increment the program counter.
2. Decode the instruction and read registers R6 and R8.
3. Compute the effective address $X + [R8]$.
4. Store the contents of register R6 into memory location $X + [R8]$.

Addressing Modes:

Addressing modes are techniques used by the CPU to determine the location of operands required for executing instructions. They define how the address field in an instruction is interpreted, enabling the CPU to fetch operands correctly. These modes are crucial for efficient memory access, program relocation, and instruction flexibility.

Types of Addressing Modes

Table 2.1 RISC-type addressing modes.

Name	Assembler syntax	Addressing function
Immediate	#Value	Operand = Value
Register	R_i	$EA = R_i$
Absolute	LOC	$EA = LOC$
Register indirect	(R_i)	$EA = [R_i]$
Index	$X(R_i)$	$EA = [R_i] + X$
Base with index	(R_i, R_j)	$EA = [R_i] + [R_j]$

EA = effective address

Value = a signed number

X = index value

Immediate Addressing

The operand is directly specified in the instruction itself. This mode is used when the value is known at the time of writing the program.

Example: `MOVE R1, #5` moves the value `5` into register `R1`.

Absolute Addressing

The operand is in a memory location, the address of this location given explicitly in the instruction.

Example: `Load R2, NUM1` ; which loads the value in the memory location NUM1 into register R2

Direct Addressing

The instruction contains the memory address of the operand. The CPU fetches the data directly from this address.

Example: `LOAD R1, 1000` loads data from memory address `1000` into register `R1`.

Register Indirect Addressing

The instruction specifies a memory location or register that holds the actual address of the operand. The CPU first fetches this address and then accesses the operand.

Example: `LOAD R1, (R2)` loads data from the memory location pointed to by register `R2`.

Register Addressing

The operand resides in a CPU register specified in the instruction. This mode is fast as it avoids memory access.

Example: `MOV R1, R2` operates directly between registers R1 and R2.

Example: `ADD R2, R0, R1` (Add values from R0 and R1, store result in R2).

Indexed Addressing Mode The address is calculated by adding an offset to a base register before accessing memory.

Example: `Load R0, 20(R5)` (Load value from memory at address $R5 + 20$ into R0).

Base Indexed addressing mode This mode combines a base address from a register and a displacement value from the instruction to calculate the effective address. It is commonly used for arrays and relative addressing. Example: Accessing an array element using `Base + Index`.

Example: `Load R0, (R1,R3)` (load value from memory at address $R1+R3$ into R0).